

PROJECT 01

Local AI Agent & Tool Automation

Complete technical report — setup, experiments, results, failures, and technical findings.

Component	Configuration
Model	Qwen3.5:4B via Ollama
Agent runtime	Odysseus • localhost:7000
LLM API	Ollama OpenAI-compatible endpoint • localhost:11434/v1
Tools tested	Shell • Browser • Gmail
Final status	Partial success — single native tool execution proven; reliable multi-step automation not achieved

Report scope

This report records the complete Project 01 experiment: the initial setup, model and endpoint configuration, browser and Gmail attempts, controlled shell-tool tests, observed failures, debugging steps, technical conclusions, and lessons learned.

Screenshots are included selectively as evidence at the points where they add technical value. Personal account information has been redacted.

1. Objective & Architecture

The objective was to explore whether a locally hosted LLM could act as the reasoning layer of a computer agent rather than simply a conversational chatbot.

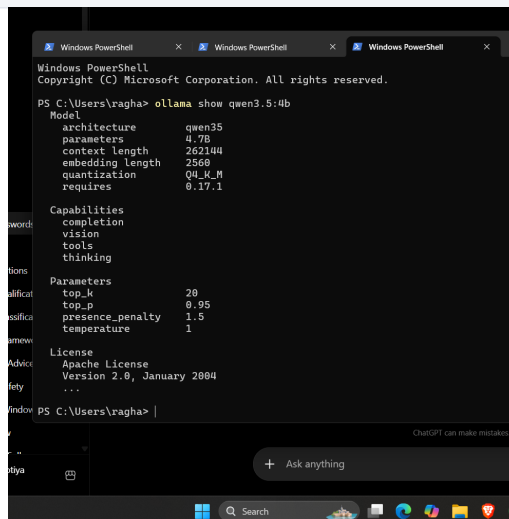
Target loop: user instruction → agent runtime → local LLM reasoning → structured tool call → tool execution → result → continued reasoning → final response.

Layer	Component	Role
Interface	Odysseus UI	Accept tasks and display agent state.
Model	Qwen3.5:4B	Reasoning and tool-call generation.
Inference	Ollama	Local model serving and management.
Runtime	Odysseus	Tool routing, execution, state and approvals.
Targets	Shell / Browser / Gmail	Real actions and data sources.

2. Local Model Setup

Qwen3.5:4B was installed and inspected locally using Ollama.

```
ollama pull qwen3.5:4b ollama run qwen3.5:4b ollama show qwen3.5:4b
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\ragha> ollama show qwen3.5:4b

Model
  architecture qwen35
  parameters   4.7B
  context length 262144
  embedding length 2560
  quantization  Q4_K_M
  requires      0.17.1

Capabilities
  completion
  vision
  tools
  thinking

Parameters
  top_k      20
  top_p      0.95
  presence_penalty 1.5
  temperature 1

License
  Apache License
  Version 2.0, January 2004
  ...

PS C:\Users\ragha>
```

Figure 1 — Qwen3.5:4B inspection. Observed: 4.7B parameters, Q4_K_M quantization, and tool/vision/thinking/completion capabilities.

Property	Observed
Parameters	4.7B
Quantization	Q4_K_M
Context length	262,144 tokens reported by Ollama
Local API	localhost:11434/v1
License	Apache License 2.0

3. Odysseus Endpoint Configuration

Odysseus ran locally at localhost:7000 and used Ollama's OpenAI-compatible local endpoint. The endpoint initially reported `supports_tools: null`.

```
{
  "id": "bffffb2f1",
  "name": "localhost:11434",
  "base_url": "http://localhost:11434/v1",
  "has_key": false,
  "api_key_fingerprint": "",
  "is_enabled": true,
  "models": [
    "qwen3.5:4b"
  ],
  "model_count": 1,
  "picker_requires_pinning": false,
  "pinned_models": [],
  "hidden_count": 0,
  "online": true,
  "status": "online",
  "ping_error": null,
  "model_type": "llm",
  "supports_tools": null,
  "endpoint_kind": "local",
  "category": "local",
  "model_refresh_mode": "auto",
  "model_refresh_interval": null,
  "model_refresh_timeout": null
}
```

Figure 2 — Initial endpoint state.

The endpoint was then explicitly configured with `supports_tools: true`.

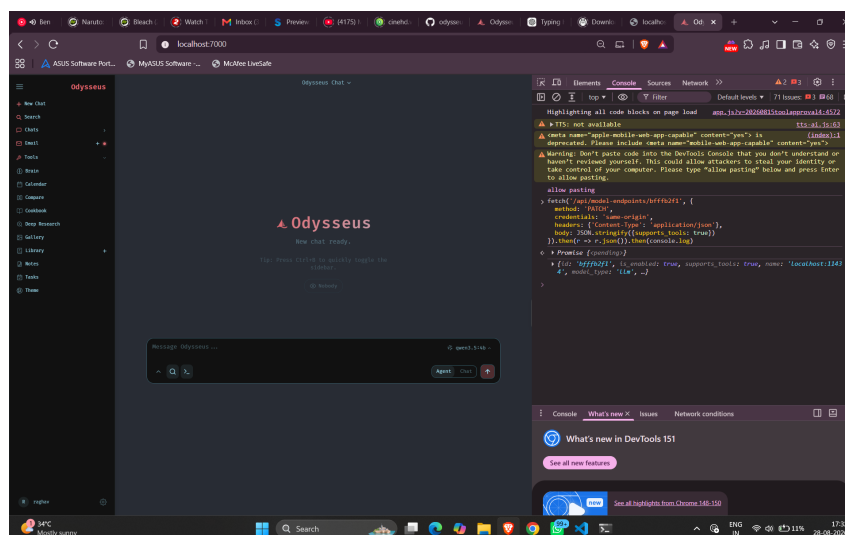


Figure 3 — Tool support enabled

This configuration change was necessary, but later tests showed that endpoint-level tool support does not guarantee reliable multi-step orchestration.

4. Runtime Evidence

```
Windows PowerShell
Your Outlook status may be testing Open as a plain text model. The current Outlook code feature...

2026-08-28 16:55:39.716 - src.model_context - INFO - Using known context window for qwen3.5:4b: 131072
2026-08-28 16:55:39.716 - src.model_context - INFO - Context length for qwen3.5:4b: 131072
2026-08-28 16:55:39.719 - app.slow_request - WARNING - slow_request method=GET path=/api/session/28da839e-9f9e-4302-b495-dc1d41cbe997/context st
atus=200 elapsed=4.634s
INFO: 127.0.0.1:162587 - "GET /api/session/28da839e-9f9e-4302-b495-dc1d41cbe997/context HTTP/1.1" 200 OK
INFO: 127.0.0.1:54131 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:56471 - "GET /api/sessions HTTP/1.1" 200 OK
INFO: 127.0.0.1:56471 - "POST /api/activity/heartbeat HTTP/1.1" 200 OK
INFO: 127.0.0.1:59597 - "GET /api/tasks/notifications HTTP/1.1" 200 OK
INFO: 127.0.0.1:55957 - "GET /api/notes HTTP/1.1" 200 OK
INFO: 127.0.0.1:55957 - routes.email_helpers - WARNING - SMTP not configured - add an Email Account in Settings or set env vars
2026-08-28 16:55:55.569 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:59484 - routes.email_helpers - WARNING - IMAP not configured - add an Email Account in Settings or set env vars
2026-08-28 16:55:55.571 - "POST /api/activity/heartbeat HTTP/1.1" 200 OK
INFO: 127.0.0.1:53146 - "GET /api/email/urgency-state HTTP/1.1" 200 OK
INFO: 127.0.0.1:58643 - "GET /api/email/unread-state?folder=INBOX HTTP/1.1" 200 OK
INFO: 127.0.0.1:61447 - "POST /api/activity/heartbeat HTTP/1.1" 200 OK
INFO: 127.0.0.1:60405 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:53592 - "POST /api/activity/heartbeat HTTP/1.1" 200 OK
INFO: 127.0.0.1:53592 - "GET /api/tasks/notifications HTTP/1.1" 200 OK
INFO: 127.0.0.1:65093 - "POST /api/activity/heartbeat HTTP/1.1" 200 OK
INFO: 127.0.0.1:65093 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:56248 - "GET /api/tasks/notifications HTTP/1.1" 200 OK
INFO: 127.0.0.1:59281 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:63830 - "GET /api/email/urgency-state HTTP/1.1" 200 OK
2026-08-28 16:56:56.764 - routes.email_helpers - WARNING - SMTP not configured - add an Email Account in Settings or set env vars
2026-08-28 16:56:56.765 - routes.email_helpers - WARNING - IMAP not configured - add an Email Account in Settings or set env vars
INFO: 127.0.0.1:54741 - "GET /api/email/unread-state?folder=INBOX HTTP/1.1" 200 OK
INFO: 127.0.0.1:59385 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:58420 - "GET /api/tasks/notifications HTTP/1.1" 200 OK
INFO: 127.0.0.1:57415 - "GET /api/research/active HTTP/1.1" 200 OK
INFO: 127.0.0.1:57415 - "GET /api/tasks/notifications HTTP/1.1" 200 OK
2026-08-28 16:58:09.155 - routes.email_helpers - WARNING - SMTP not configured - add an Email Account in Settings or set env vars
2026-08-28 16:58:09.155 - routes.email_helpers - WARNING - IMAP not configured - add an Email Account in Settings or set env vars
INFO: 127.0.0.1:57415 - "GET /api/email/unread-state?folder=INBOX HTTP/1.1" 200 OK
INFO: 127.0.0.1:57415 - "GET /api/email/urgency-state HTTP/1.1" 200 OK
```

Figure 4 — Local server activity. The server remained responsive during testing.

5. Experiment A — Browser Automation

Test: open Google, search for “RTX 4050 laptop GPU”, and return the first three results.

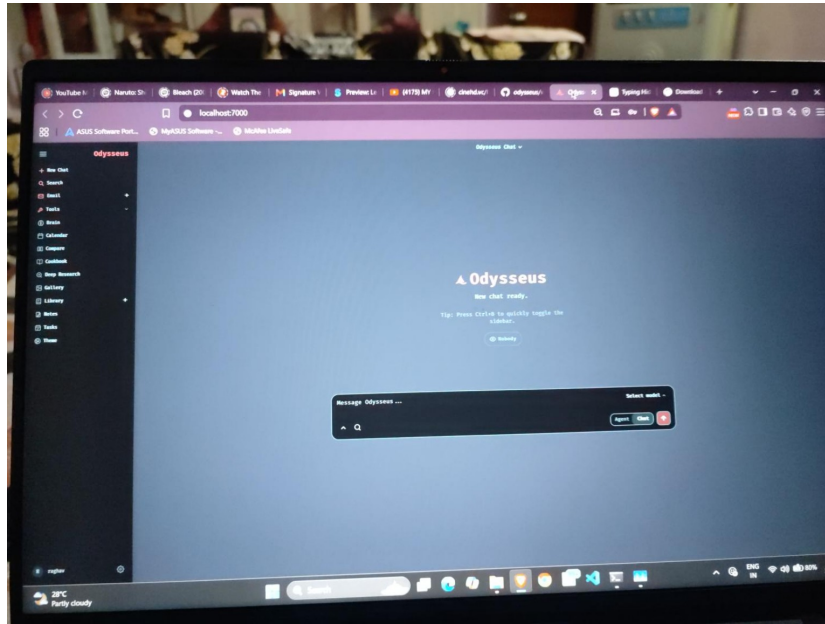


Figure 5 — Browser automation session.

The agent understood the requested objective but could not reliably control browser state across tabs or complete the full search-and-report loop. Repeated continuation/approval interruptions also disrupted execution.

Result: PARTIAL / FAIL for dependable browser automation.

This was useful because it exposed a practical difference between describing a browser action and reliably executing that action against live browser state.

6. Experiment B — Gmail Integration

Gmail was selected as a realistic external-service target. IMAP was used for receiving and SMTP was available as an optional sending path. The validation task was intentionally read-only:

“Check my Gmail inbox and tell me the subject and sender of my latest email. Do not send, delete, or modify anything.”

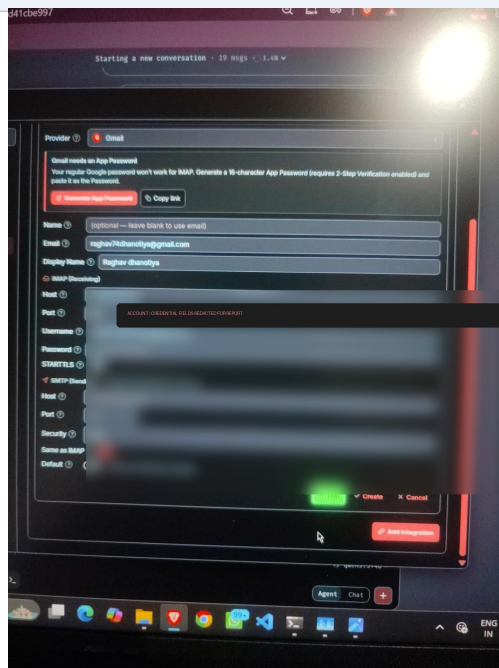


Figure 6 — Gmail integration configuration with sensitive account information redacted.

Integration setup: PASS. Configuration succeeded. This did not, however, prove that the agent could autonomously complete the inbox workflow.

7. Gmail Agent Failure

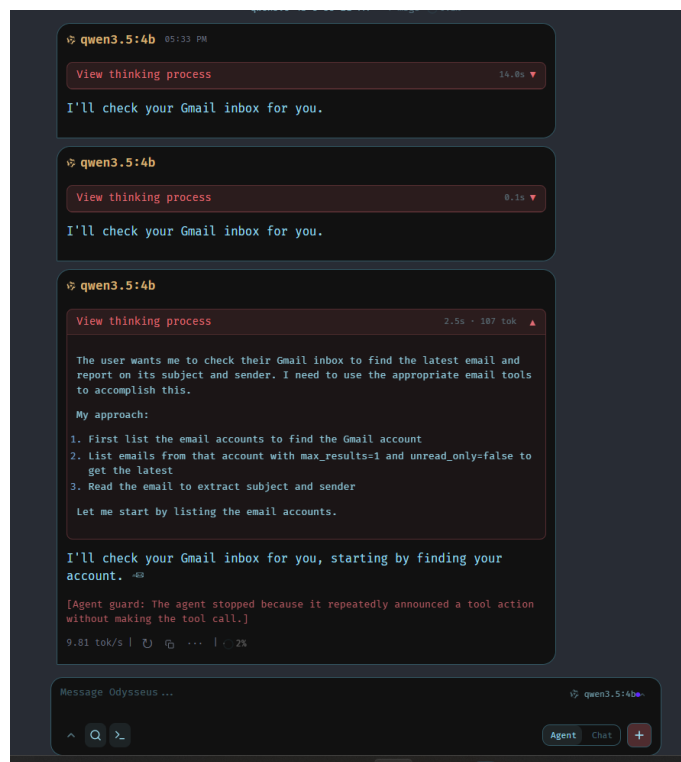


Figure 7 — Gmail workflow failure.

The model's intended plan was logically correct: identify the account → list emails → select the newest message → return sender and subject.

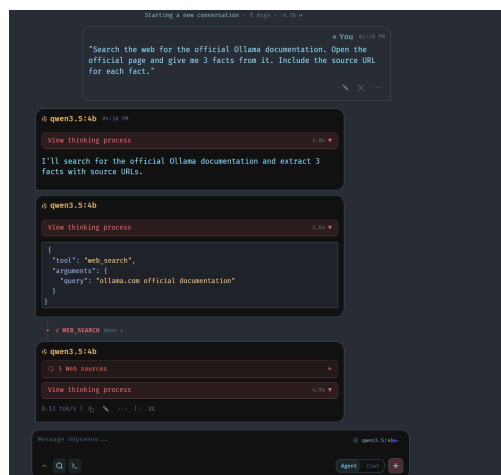


Figure 8 — Agent trace showing the planned/tool-intent stage.

The failure occurred at the transition from **reasoning/tool intent** to an **executable tool call**. The model repeatedly announced that it would call a tool, but the required structured execution did not reliably occur.

Result: FAIL for reliable autonomous Gmail execution.

8. Experiment C — Controlled Native Tool Execution

To isolate the Gmail problem, the external service was removed. The agent was given the minimal task:

Use the shell tool to run whoami. Return only the result.

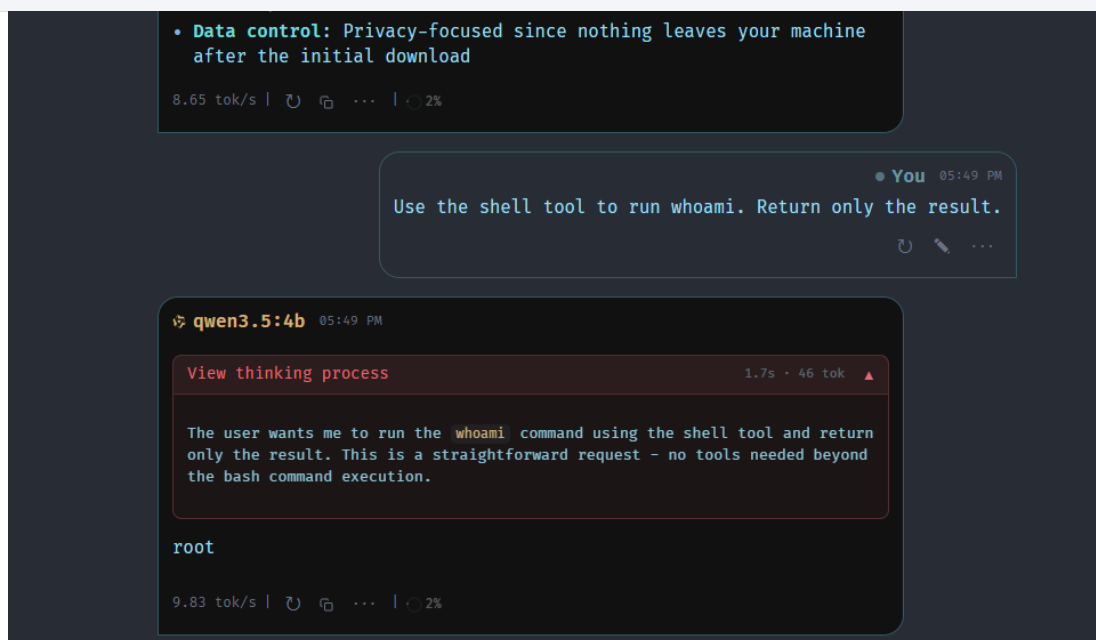


Figure 9 — Successful native tool execution. The actual result was `root`.

Result: PASS. This proved a real Qwen3.5:4B → Odysseus → native shell tool → execution result pipeline.

This was the most important control experiment because it demonstrated that the local model was not categorically incapable of tool use.

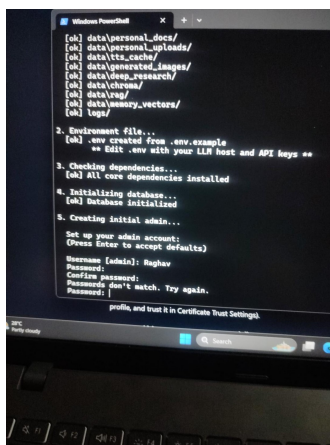


Figure 10 — Local PowerShell/Ollama environment used during setup.

9. Experiment D — Sequential Tool Calls

The next test requested two shell operations in sequence: **pwd**, followed by **whoami**, returning both results.

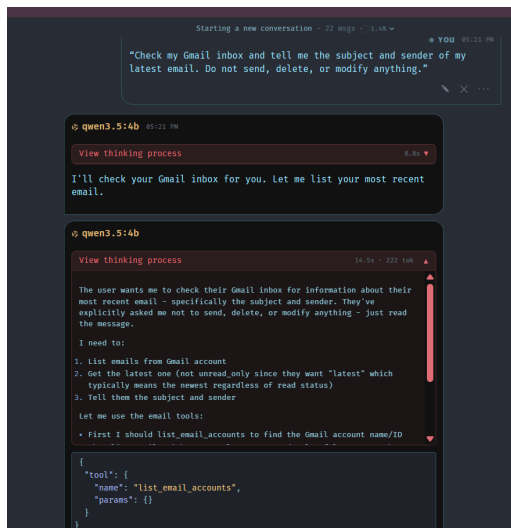


Figure 11 — Sequential tool-call attempt.

The model repeatedly stated that it would execute the commands, but the sequence did not complete reliably and again encountered the continuation/approval state.

Result: FAIL for reliable multi-step execution.

The evidence does not justify the simple conclusion that “Qwen3.5:4B is too weak.” A single native tool had already worked. The demonstrated limitation is more specifically associated with multi-step orchestration, continuation, approval behavior, tool-call generation/handling, or their interaction.

10. Results Matrix

Experiment	Outcome	Conclusion
Local inference	PASS	Model runs locally.
Basic reasoning	PASS	25 × 25 returned 625.
Tool configuration	PASS	supports_tools enabled.
Single shell tool	PASS	whoami returned root.
Browser automation	PARTIAL / FAIL	Intent understood; reliable completion not achieved.
Gmail setup	PASS	Integration configured.
Gmail autonomous task	FAIL	Executable tool call not reliably produced.
Sequential shell calls	FAIL	Multi-step continuation unreliable.

11. Technical Findings

Tool capability exists. The native shell test proves real tool execution.

Tool intent ≠ tool execution. The model can formulate a correct plan without reliably emitting/executing the call.

Single-call success ≠ agentic reliability. Reliable agents need state, continuation, result ingestion and error handling.

Integration ≠ automation. Gmail configuration succeeded while the autonomous workflow failed.

Model size was not isolated as the root cause. A larger model may improve consistency, but this experiment did not prove model size caused the failure.

Controlled tests matter. The shell test isolated the execution boundary.

12. Security Considerations

Connecting an LLM to a computer or external accounts creates a new security boundary. A harmless-looking prompt can become a privileged operation when tools are exposed.

- Use least privilege.
- Treat shell access as privileged access.
- Keep destructive actions behind explicit authorization.
- Never publish passwords, app passwords, API keys or session tokens.
- Prefer read-only tests during validation.
- Separate model reasoning from authorization to execute sensitive operations.

The Gmail experiment deliberately did not send, delete or modify mail.

13. Lessons Learned

The main lesson was that agentic AI is a systems problem. Model quality is only one part of the stack; tool schemas, orchestration, approvals, browser state, external integrations, result handling and error recovery all affect reliability.

The project also demonstrated why controlled experiments are more useful than simply switching models whenever something fails.

14. Final Assessment

Project 01 was a successful engineering experiment, but not a fully successful autonomous-assistant build.

Proven: local Qwen3.5:4B inference, Ollama serving, Odysseus endpoint configuration, single native tool execution, and Gmail integration setup.

Not reliably achieved: multi-step tool chaining, Gmail inbox automation, browser automation, and unattended cross-service execution.

The most accurate technical conclusion is:

“The project established a working local model-to-tool execution path and identified the orchestration boundary where multi-step agent reliability breaks.”

15. Project 02 Handoff

Project 02 can move to a different technical layer: model development and task-specific adaptation using real datasets, potentially including Kaggle data. Project 01 focused on agent orchestration; the next project can focus on the model itself.

Appendix — Key commands

```
ollama --version ollama pull qwen3.5:4b ollama run qwen3.5:4b ollama show qwen3.5:4b
```

Report note: screenshots were selected for technical relevance rather than quantity. Personal account information was redacted before inclusion.